Developer Instructions

Services

Two services: the main app (8089) and the **games microservice** (`games-service/` , 8095). The page loads the games menu from the games service (`/api/config` tells the page its URL; the games service allows only the main app's origin by CORS). Optional Kong (`kong/` , `docker-compose.kong.yml`) can front both. Flutter / Spring AI / Kong notes are at the end.

Architecture (V2)

```
browser (Edge)  --  /                       index.html + js/{visual-
errors,speech,app,hud,games-menu}.js + school/{school-common,panel}.js
                --  /api/v1/documents/analyze-sync        PDF -> JSON (virtual
thread, 5 min limit)
                --  /api/v1/translate  (+ /status, /selftest)   Ollama first (if it
answers in the right script), else the online translator
                --  /api/error-guides                      the visual error catalogue
(colour, scene, reason, steps, idea)
                --  /error  and exception resolver         HD animated HTML error
pages for browsers, JSON for fetch()
                --  /school/*.html                         pop-outs: admin, test,
live class, games
                --  /school/api/{auth,admin,student}/...   School REST (session login
+ CSRF token), /school/files/material/{id}
```

Packages in `com.org.pdfplatform` : `controller` , `pdf` , `translate` (OllamaClient, WebTranslateClient, PromptBuilder, TranslationService, ScriptDetector), `service` , `model` , `exception` , `launcher` , `config` , **`school`** (`model` JPA entities `sch_*` , `repo` , `service` , `testing` , `web`).

Translation engines

`TranslationService.translate` : `translate.prefer` = `auto` (default), `ollama` or `web` . With `auto` : Ollama if running; its answer is checked by `ScriptDetector.outputMatchesTarget` (Indic/Urdu targets must come back in the right script, Spanish/French must differ from the input). A wrong or failing answer falls back to `WebTranslateClient` (Google POST -> Google GET in small pieces -> MyMemory); if all fail the error lists every reason. `translate.web.enabled=false` keeps text on the PC. `GET /api/v1/translate/selftest?target=hi` runs every route and reports; the page's **Test translation** button calls it.

Visual error system

- `ErrorCodeModel` (server codes) + `ErrorGuide` (catalogue: category, emoji, headline, reason, steps, tip, colours; also browser-only codes such as NETWORK, NO_VOICE).
- JSON: `ErrorResponseModel` now carries `guide` (filled from `ErrorGuide`). `GlobalExceptionHandlerModel` answers JSON; `handleFramework` maps 404/405/415/400 and

`SchoolException`.

- Pages: `VisualErrorResolver` (HandlerExceptionResolver, highest precedence) renders `ErrorPageRenderer` HTML when the request wants HTML (Accept text/html, not cors); `VisualErrorController` replaces the whitelabel `/error`.
- Browser: `js/visual-errors.js` renders the card from the server catalogue (fallback built in) and adds an *instance tip* from the situation (button, language, voice, file size, page).
- To add an error: add the enum value, a `g("CODE", "CATEGORY", emoji, headline, reason, step, tip, ...)` entry in `ErrorGuide`, throw it (or `SchoolException`). `ErrorGuideTest` fails if a code has no complete guide.

School module

- **DB:** `application.yml` -> Aiven MySQL; password in `config/db-secret.properties`. Hikari does not fail at start (`initialization-fail-timeout: -1`), Hibernate does not touch the DB at start (`ddl-auto: none`, `allow_jdbc_metadata_access: false`). `SchoolDb.ensureReady()` runs `school/schema.sql` (`CREATE TABLE IF NOT EXISTS sch_*`) and seeds the school record + first admin on the first School request and retries every 20 s. Every table has a primary key (Aiven requires it). Other tables in the same database are never touched.
- **Auth:** `SchoolSession` (server session, BCrypt via spring-security-crypto - no Spring Security, so the old pages stay open), `SchoolAuthInterceptor` guards `/school/api/**` and `/school/files/**` (DB ready, login, admin vs student area, `X-CSRF-Token` on every change). Login is throttled (5 wrong tries / 10 min / user name).
- **Tests:** `TestService.save` validates, writes `Test<DD-MM-YYYY>.properties` + `.json` (`school.tests-dir`, copy kept in `sch_tests.properties_text`), second test of a day gets `_2`. `TestPropertiesCodec` is the single reader/writer; correct answers never reach the student's browser (`/start` sends options only). `Grader`: exact match of the ticked boxes.
- **Student test flow:** `/tests/{id}/start` (only on the test date, only assigned students, timer from first open, resumes) -> `/submit` (blank if > 90 s after time) -> `/result`.
- **Live class:** `LiveService`; the teacher sets the shared page, students poll `/student/live/state` every 2 s; chat messages stored in `sch_live_messages`.
- **Games:** `school/games/edu-games.js` (pure, unit tested: question generators per class band, scramble, memory pairs, quiz bank) + `game-common.js` (HD confetti canvas, scores via `/student/games/score`).
- **Panel:** `school/panel.js` fills `<aside id="schoolPanel">`; `body.school-open` widens the dock and the reader (`.shell`) shrinks. `window.WN_loadFile(file)` hands a school PDF to the normal Analyze flow.

Compass (`js/hud.js`)

Sources, first one to deliver wins: iOS `webkitCompassHeading`; Chromium `AbsoluteOrientationSensor` (reference frame *screen*); `deviceorientationabsolute`; `deviceorientation` only when `absolute===true`. Tilt and screen rotation handled (flat: where the top edge points; upright: where the back faces). `unwrap()` keeps the dial turning the short way across north (the old build spun backwards at 359 -> 1). No sensor: the person sets where the screen faces (stored in `localStorage wn.compass.facing`) or taps where the Sun is; the Sun marker is computed from time + browser location (`sunPosition`, within 0.03 degrees of astral in tests).

How speech works

`js/speech.js` (`EdgeSpeech`): `ready()` waits for Edge's asynchronous voice list; `pickVoice()` ranks installed voices by locale -> chosen name -> "Natural"; `speak()` cuts the text into <= 220 character chunks, sets `utterance.voice`, watches `onstart` (6 s) and tries the next voice before reporting a plain-language error.

Configuration (`application.yml`)

`server.port` | `pdf-voice.*` | `translate.prefer/web.enabled/ollama.*` | `app.auto-open-browser`, `app.browser-isolated-profile` | `spring.datasource.*` | `school.zone/tests-dir/show-answers-after-submit/admin.*` | env: `SPRING_DATASOURCE_PASSWORD`, `ADMIN_INITIAL_PASSWORD`, `DB_HOST/PORT/NAME/USER`.

Adding a game (in `games-service`)

Put `mygame.html` in `games-service/src/main/resources/static/games/` (copy a shell, `GK.start({id,title,tip,reset,update(dt),draw(ctx,w,h),hud})`, `GK.gameOver()`), add a `GameInfo` to `GameCatalogService`. For a School game add the page under `static/school/games/`, its id to `GameScoreService.GAMES` and to the panel list.

Tests

`mvn test` (Java) and `cd js-tests && npm install && npm test` (73 tests: speech, page with simulated Edge, compass incl. north crossing / sensor priority / manual facing / Sun calibration, visual errors, game logic). Pure JUnit tests (`TestPropertiesCodecTest`, `ErrorGuideTest`, `ErrorPageRendererTest`) need no Spring.

Flutter + Spring AI + Kong for text-to-voice - does it change anything?

Short answer: **not for the voice**. The voice is made by the device's speech engine, not by the backend. Spring AI (translation) and Kong (routing, rate limits) sit behind that and do not change how a voice sounds. Flutter (`flutter_tts`) only wraps the phone's/PC's own engines: Android's Google voices, iOS voices, Windows voices - as far as I know the Edge *Natural* voices are available to web pages in Edge but not to ordinary Windows apps. So a Flutter Windows app would not get today's Edge voices; Flutter Web in Edge would (same browser engine as now). Flutter *is* worth it for an installable Android/iOS app (offline phone voices, lock-screen controls, store distribution). For Razorpay on the SuryaHomeStay site, the web checkout already works on phones; Flutter (`razorpay_flutter`) matters only if you want a store app.

Verification (honest)

Run by me: all 73 browser tests; 18 pure JUnit tests (executed with a small JUnit stand-in, results identical to the assertions in the files); the `.properties` round trip with Kannada text; the translation output check; the Sun azimuth against an independent astronomy library; every error page rendered in headless Chromium (HTML well-formed, text escaped, looked at); the panel, admin console, test, live class and all four games clicked through in headless Chromium against a mocked School API (no JavaScript errors). **Not run:** Maven (not available offline there) - the Spring-level JUnit tests (services, login, interceptor, error resolver) were only compile-checked with a custom lint; the Spring context, JPA mappings and

`school/schema.sql` have **never run against MySQL**; no real Aiven connection; nothing heard through a real Edge or tried against a real Ollama / the online translators. Run `Build_all.bat` and `mvn test` first and send me the first error if any.